MEPFS Budget Model: Data, Features, Formulas, and Training

Process overview (part-by-part)

Part 1: Data acquisition and schema normalization (CSV import, header cleaning).
Part 2: Data cleaning and target transformation (NaN handling, $\log(1 + \cdot)$).
Part 3: Feature engineering (parsing building scale, aggregating subsystem costs, creating normalized cost-per-unit metrics).
Part 4: Feature matrix construction (8 raw engineered inputs).
Part 5: Polynomial expansion (degree 2) and standardization (z-score).
Part 6: Train/test split (80/20 holdout).
Part 7: Baseline model training (LSBoost) and evaluation.
Part 8: Feature selection via ensemble importances (median threshold).
Part 9: Hyperparameter optimization (Bayesian) and final model training.
Part 10: Final evaluation, artifacts saving, and inference pipeline.

Variables glossary (concise definitions)

Clean name	Source header	Type	Unit	Definition / Role
project	Project	text	n/a	Project title; used to parse `num_storeys`, `num_classrooms` via regex.
year	Year	numeric	year	Project year; raw feature.
budget	Budget	numeric	PHP	Target variable; filtered to >100,000 then log-transformed.
`num_storeys`	parsed from project	numeric	count	Integer before "STY"; mean-imputed if missing.
`num_classrooms`	parsed from project	numeric	count	Integer before "CL"; mean-imputed if missing.
Derived MEPFS Aggregates & Ratios				
`total_electrical_cost`	sum of electrical columns	numeric	PHP	Sum of costs for panelboards, lighting, conduits, and wires.
`total_plumbing_sewer_cost`	sum of plumbing columns	numeric	PHP	Sum of costs for sewer lines and plumbing fixtures.
`electrical_cost_per_classroom`	derived	numeric	PHP/classroom	`total_electrical_cost` / `num_classrooms`. Captures electrical cost intensity.
`plumbing_cost_per_classroom`	derived	numeric	PHP/classroom	`total_plumbing_sewer_cost` / `num_classrooms`. Captures plumbing cost intensity.
`fire_alarm_cost_per_classroom`	derived	numeric	PHP/classroom	Cost of fire alarm system / `num_classrooms`. Captures safety system cost intensity.

Final raw feature set (8 inputs) and counting proof

The MEPFS training script fixes the raw input vector to the following 8 features:

$X_{\text{raw}} = [\,\text{year},\ \text{num\_storeys},\ \text{num\_classrooms},\ \texttt{total\_electrical\_cost},\ \texttt{total\_plumbing\_sewer\_cost},\ \texttt{electr}$

Count: $p = 8$. Degree-2 polynomial design yields exactly

$$\underbrace{p}_{\text{originals}} + \underbrace{p}_{\text{squares}} + \underbrace{\binom{p}{2}}_{\text{pairwise interactions}} = 8 + 8 + 28 = 44 \text{ features.}$$

Why exactly 8 input features? Justification of Feature Engineering

The 8 features were engineered to capture project scale, subsystem costs, and normalized intensity metrics, providing a robust basis for the model while avoiding multicollinearity from raw, disaggregated cost items.

$$\underbrace{1}_{\text{year}} + \underbrace{2}_{\text{parsed}} + \underbrace{2}_{\text{aggregates}} + \underbrace{3}_{\text{ratios}} = 8.$$

- **Year (1):** `year` accounts for temporal variations like inflation.
- **Parsed from Project (2):** `num\_storeys` and `num\_classrooms` are high-level proxies for the building's physical scale.
- **Aggregated Costs (2):** `total\_electrical\_cost` and `total\_plumbing\_sewer\_cost` consolidate individual line items into stable, high-level predictors for major subsystems. This reduces noise and redundancy.
- **Normalized Ratios (3):** The `..._per_classroom` features are crucial. They normalize subsystem costs by the project's programmatic size (`num_classrooms`), creating features that measure cost *intensity*. This helps the model generalize across projects of different sizes by learning the typical cost per functional unit, rather than just total cost.

Polynomial/interaction expansion and scaling

We build degree-2 features via `generatePolyFeatures(T,2)` and compute z-scores on the training split only:

$$\mu_j = \frac{1}{n}\sum_i X_{ij}, \quad \sigma_j = \sqrt{\tfrac{1}{n-1}\sum_i (X_{ij}-\mu_j)^2}, \quad Z_{ij} = \frac{X_{ij}-\mu_j}{\sigma_j},\ \text{NaN} \to 0.$$

Train/test split and targets

We hold out 20% using `cvpartition(HoldOut=0.2)`. The response is $y = \log(1 + \text{budget})$; predictions are inverted with $\exp(\cdot) - 1$ for reporting in PHP.

Baseline and optimized models

Baseline uses LSBoost with `NumLearningCycles=250` on the scaled 44 features. **Feature selection** keeps features with importance above the median (per `predictorImportance`). **Optimized model** is LSBoost with hyperparameters tuned via Bayesian optimization over: cycles [500,1000], splits [8,32], learn rate [0.01,0.1] (log-scale).

Figures (testing evidence)

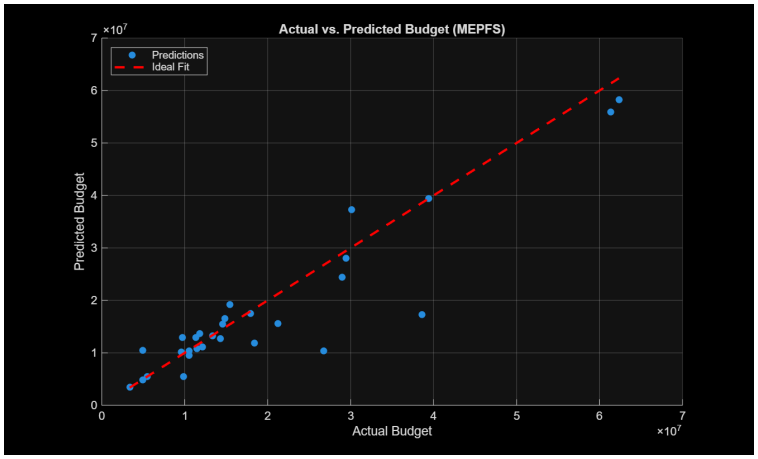

Figure 1: Actual vs. Predicted (held-out test).

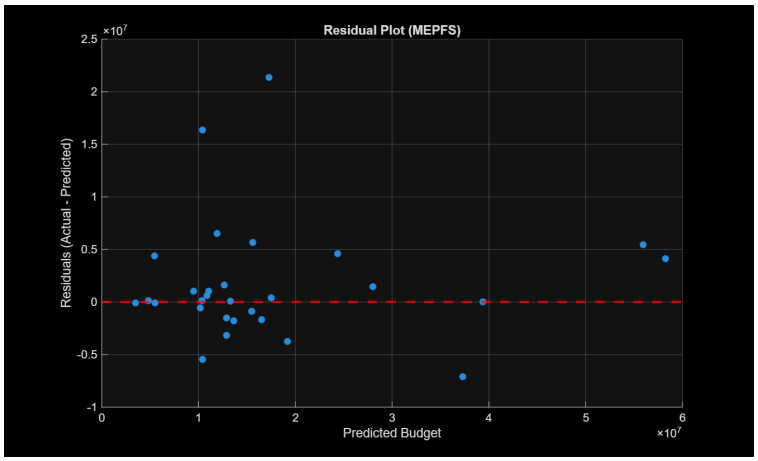

Figure 2: Residuals vs. Predicted.

Testing metrics (numerical summary)

To document exact test-set performance, we include an auto-generated snippet if available. (Run a script to generate and include the exact metrics.)

Exact numbers behind feature-importance

For tree ensembles like the LSBoost model used here, feature importance is calculated by summing the reductions in Mean Squared Error (MSE) at every split in every tree where that feature is used. The reduction is weighted by the number of observations at the node being split. This method quantifies how much each feature contributes to improving the model's predictive accuracy.

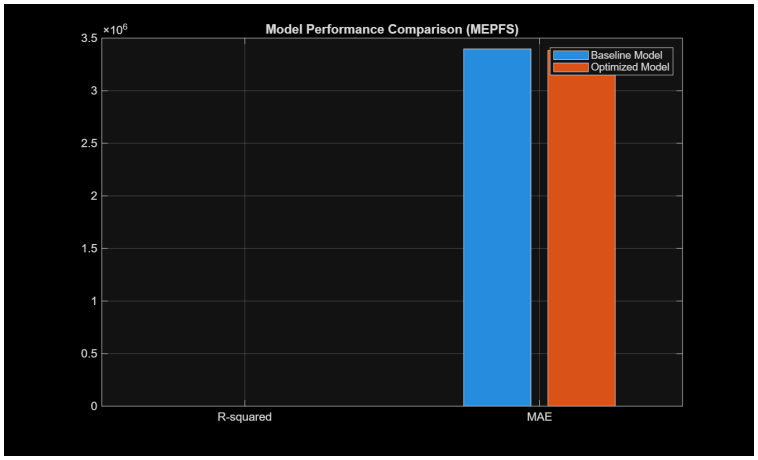

Figure 3: Baseline vs. Optimized performance (R^2, MAE) on test split.

The formula for the importance of a single feature j is:

$$\text{Imp}(j) = \sum_{t \in \text{trees}} \sum_{n \in \mathcal{N}_{t,j}} (\Delta\text{MSE}(n) \times w(n))$$

where:

- $\mathcal{N}_{t,j}$ is the set of nodes in tree t that use feature j for a split.
- $\Delta\text{MSE}(n)$ is the reduction in MSE achieved by splitting node n.
- $w(n)$ is a weight representing the number of training samples at node n.

The bar chart below visualizes these raw importance scores for the top-20 features from the final optimized model, sorted in descending order.

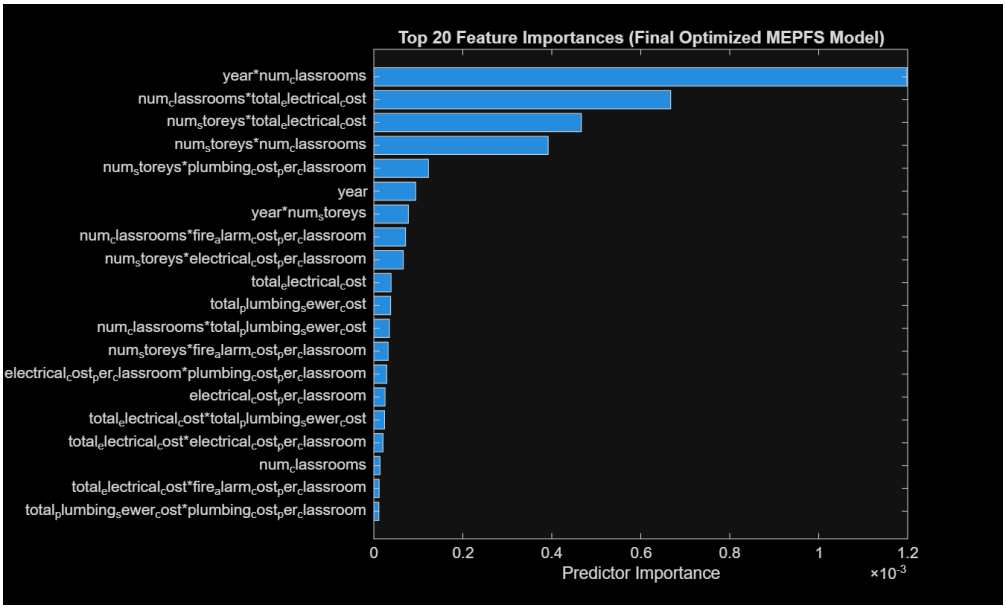

Figure 4: Top-20 feature importances of the optimized MEPFS ensemble.

If generated, the exact table will be included below: (Run a script to export and include exact values.)

Model Architecture: Why No Hidden Layers?

The model used for MEPFS budget prediction is a **Least-Squares Boosting (LSBoost) ensemble**, which is a powerful machine learning technique based on decision trees. It is fundamentally different from a neural network and, as a result, **does not have hidden layers**. This section explains the model's structure, its parameters, and the importance of this architectural choice.

Architectural Parameters (The Analog to "Layers")

Instead of layers, weights, and activation functions, the complexity of an LSBoost model is defined by its component trees. The key architectural parameters are:

- **Number of Trees (`NumLearningCycles`):** This is the total number of decision trees in the ensemble. Each tree is trained sequentially to correct the errors of the one before it. This is the most direct analog to the "depth" or "size" of a neural network.
  - **Baseline Model:** Uses a fixed number of **250 trees**.
  - **Optimized Model:** The number of trees is automatically tuned by searching for the best value within the range of **500 to 1000 trees**.
- **Tree Depth (`MaxNumSplits`):** This parameter controls the complexity of each individual tree by limiting how many decision points (splits) it can have.
  - **Baseline Model:** Uses the default value of **10 splits**.
  - **Optimized Model:** The depth is tuned by searching for the best value between **8 and 32 splits**.

Justification for Hyperparameter Ranges in the Optimized Model

The optimized model does not have a single, fixed number for its parameters because it uses **Bayesian optimization**. This is an intelligent and automated process for finding the best-performing combination of hyperparameters.

- **Search Space:** We define a *range* of possible values for each parameter (e.g., '[500, 1000]' for trees).
- **Automated Search:** The algorithm iteratively trains different model versions, using the results from past attempts to intelligently decide which combination to try next. This is far more efficient than manual trial-and-error.
- **Result:** The final "optimized" model is the one that performed best during this search. Its exact number of trees and splits are an *output* of the training process itself, not a predetermined input. The ranges specified here represent the boundaries of that search.

Why This Architecture is Important

Choosing a tree-based ensemble like LSBoost over a neural network offers several key advantages for this specific problem:

- **Interpretability:** Unlike the "black box" nature of neural networks, tree models provide clear **feature importances**. As shown in Figure **??**, we can see exactly which inputs (e.g., `total_electrical_cost`, `num_classrooms`) are the most influential drivers of the budget. This is crucial for understanding the cost structure and building trust in the model's predictions.
- **Performance on Tabular Data:** For structured, tabular data like this project's dataset, tree-based ensembles often outperform neural networks. They are highly effective at capturing complex, non-linear interactions between features.
- **Robustness:** The boosting process, where each tree corrects the errors of its predecessors, makes the model robust to noise and outliers in the data.

Inference pipeline (single row)

1. Normalize field names; compute aggregate costs and per-classroom ratios.
2. Order to final 8 features; build degree-2 features; align to training names.
3. Standardize with saved μ, σ; apply selected feature mask; predict in log-space; invert with $\exp(\cdot) - 1$.